

MetaDSE: A Few-shot Meta-learning Framework for Cross-workload CPU Design Space Exploration

Runzhen Xue^{1,2,*}, Hao Wu^{3,*}, Mingyu Yan^{1,2,†}, Ziheng Xiao¹, Xiaochun Ye^{1,2}, Dongrui Fan^{1,2}

¹State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China;

²University of Chinese Academy of Sciences, Beijing, China;

³University of Electronic Science and Technology of China, Chengdu, China;

{xuerunzhen21s, yanmingyu, xiaoziheng, yexiaochun, fandr}@ict.ac.cn, wh.pyjngd@gmail.com

Abstract—Cross-workload design space exploration (DSE) is crucial in CPU architecture design. Existing DSE methods typically employ the transfer learning technique to leverage knowledge from source workloads, aiming to minimize the requirement of target workload simulation. However, these methods struggle with overfitting, data ambiguity, and workload dissimilarity.

To address these challenges, we reframe the cross-workload CPU DSE task as a few-shot meta-learning problem and further introduce MetaDSE. By leveraging model agnostic meta-learning, MetaDSE swiftly adapts to new target workloads, greatly enhancing the efficiency of cross-workload CPU DSE. Additionally, MetaDSE introduces a novel knowledge transfer method called the workload-adaptive architectural mask algorithm, which uncovers the inherent properties of the architecture. Experiments on SPEC CPU 2017 demonstrate that MetaDSE significantly reduces prediction error by 44.3% compared to the state-of-the-art. MetaDSE is open-sourced and available at this anonymous GitHub.

Index Terms—Design Space Exploration, Cross-workload, Prediction Model, Meta-learning

I. INTRODUCTION

Design Space Exploration (DSE) is pivotal in modern CPU architecture design, enabling the optimization of configurations to balance multiple objectives like performance, power, and area (PPA) [1], [2]. The increasing complexity of CPU architectures has led to an exponential expansion of the design space, presenting significant challenges in efficiently identifying optimal configurations.

Prior DSE frameworks [3]–[11] have utilized learning-based surrogate models to expedite the DSE process, achieving substantial time savings compared to exhaustive simulations. However, these workload-specific DSE frameworks rely heavily on extensive simulations to generate sufficient data for training machine learning models, necessitating the evaluation of numerous design points. This approach can be exceedingly time-intensive, especially for complex architectural spaces.

To mitigate this challenge, various studies [12]–[18] have adopted the transfer learning technique to reduce simulation overhead. By leveraging previously collected data from source workloads, transfer learning enables the model to adapt to unseen target workloads, a process known as cross-workload DSE. Cross-workload DSE generally follows a two-stage

training process: first, pre-training on data from source workloads, and then fine-tuning on limited data from the target workload. However, current cross-workload DSE frameworks have notable limitations in both upstream pre-training stage and downstream adaptation stage:

- **Upstream Pre-training Issues:** Existing methods typically rely on machine learning based surrogate models [12]–[14], [17], [18] combined with standard supervised learning, which are prone to overfitting the pre-training data. Additionally, the inherent data ambiguity across multiple workloads introduces optimization challenges for transfer learning, often hindering effective convergence.
- **Downstream Adaptation Issues:** Given the high cost of simulations, training samples for new workloads are often limited, leading most existing frameworks to rely on knowledge transfer for adaptation. However, these approaches typically assume workload similarity as a prerequisite [12], [15], [16], which is not always the case. When this assumption fails, the adaptation performance degrades significantly, limiting the effectiveness of traditional transfer learning methods in cross-workload CPU DSE tasks.

To tackle these issues, we rethink cross-workload CPU DSE as a few-shot problem and propose MetaDSE, an open-source framework featuring two key stages: upstream pre-training and downstream adaptation. **For the upstream pre-training stage**, we adopt a meta-learning approach [19]–[21], well-suited for addressing few-shot problems. Unlike normal supervised learning, meta-learning emphasizes “learning how to learn” across tasks, making it ideal for tackling data ambiguity in DSE. By treating each workload as a distinct task during meta-training, the model avoids overfitting to specific instances and instead captures shared patterns and relationships across tasks. **For the downstream adaptation stage**, we design an adaptation algorithm called the Workload-adaptive Architectural Mask (WAM). WAM provides a novel approach to knowledge transfer, emphasizing the inherent properties of the architecture rather than relying on workload similarity.

To summarize, we list our contributions as follows:

- We conduct a comprehensive analysis of existing CPU cross-workload DSE frameworks, identifying their limi-

*Runzhen Xue and Hao Wu contributed equally to this work.

†The corresponding author is Mungyu Yan (yanmingyu@ict.ac.cn).

tations in handling data ambiguity and enabling efficient knowledge transfer.

- We reframe the CPU cross-workload DSE task as a few-shot meta-learning problem, using the Model-Agnostic Meta-Learning (MAML) algorithm to enhance model adaptability with limited samples from new workloads.
- We introduce a novel adaptation algorithm named WAM, which captures inherent properties of the architecture, enabling efficient and generalized DSE framework across diverse workloads.
- Extensive experiments on the SPEC CPU 2017 benchmark suite show that our proposed MetaDSE framework significantly outperforms state-of-the-art methods, achieving 44.3% reductions in prediction errors across various workloads.

II. BACKGROUND & RELATED WORK

A. Cross-workload DSE framework

1) *Taxonomy*: Most cross-workload CPU DSE methods [12]–[18] use transfer learning [22], [23] to reduce sample requirements for new workloads by leveraging source knowledge, typically following three strategies. **Linear Fitting**. Work [18] uses linear regression to combine predictions from fixed source models, mapping them to the target workload label space using a few labeled samples. It merely builds a linear mapping between the source and target label spaces, relying heavily on the unrealistic assumption of identical distributions and linearity. **Data Augmentation**. Work [17] employs the Gaussian mixture model [24] to model the original data and perform augmentation. By swapping the coefficients of high-probability and low-probability Gaussian components, the method enhances rare data distributions, aiming for more balanced data augmentation while preserving the overall distribution. To some extent, this work stems from the premise of similarity in the distribution of workloads. **Similarity Analysis**. Studies [12]–[16] explore workload similarity for model adaptation using distinct strategies. [15], [16] leverage workload signatures, representing workloads during pretraining and identifying the most similar signature for downstream adaptation. This approach transfers knowledge through signature-based representation. In contrast, methods like TrEnDSE [12], TrDSE [13], and TrEE [14] adopt a data-driven approach. TrEnDSE uses Wasserstein distance [25] to measure workload similarity, while TrDSE clusters workloads using orthogonal array (OA) sampling and distributional features. TrEE further refines this with an OA-based foldover sampling strategy. These methods select similar samples from the pre-trained dataset to enhance downstream adaptation, directly integrating relevant upstream data without relying on explicit signature representation.

2) *Re-evaluating Similarity-based Approaches*: In this section, we examine the frameworks of incorporating data from source workload into downstream adaptation. One of the earliest works to apply this concept is TrDSE [13], which influenced subsequent frameworks such as TrEE [14] and

TrEnDSE [12]. However, these methods always include a one-time training process using data from the target workload. Actually, concerns over data leakage make it uncommon and impractical to access pre-trained data in many real-world scenarios. These methods can be better described as data augmentation rather than transfer learning [23].

B. Few-shot Problem & Meta-learning

The few-shot learning problem involves training models to generalize effectively using only a small number of labeled samples [26], [27], typically just a few dozen. This scenario is common in situations where labeling data is costly. To address this challenge, meta-learning has emerged as a powerful approach. In meta-learning [20], [20], a model is trained across a diverse set of tasks and is optimized to generalize well over a distribution of tasks. Here, each task is associated with a dataset $\mathcal{D}$ containing features and labels, where the entire dataset acts as a single data sample. The goal of meta-learning can be formalized similarly to supervised learning, with the optimal model represented as $\theta^* = \arg \min_{\theta} \mathbb{E}_{\mathcal{D} \sim p(\mathcal{D})} [\mathcal{L}_{\theta}(\mathcal{D})]$.

As depicted in Fig. 1, the meta-learning model is trained on a variety of curated tasks from different datasets, where the supervised signal from these tasks guides the model to perform well across a wide range of unseen tasks. One common meta-learning technique is MAML [19], which aims to learn an optimal initialization of model parameters. This initialization enables the model to rapidly adapt to any new task with just a few gradient updates, providing efficient solutions for downstream few-shot problems.

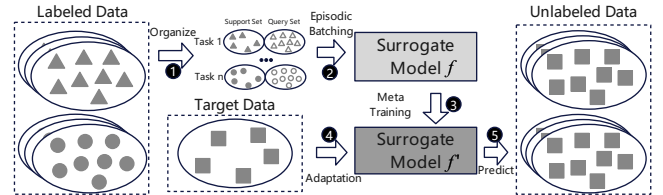


Fig. 1. Typical workflow of Model-Agnostic Meta-Learning.

III. MOTIVATION

A. Limitations in Prior Arts

Current cross-workload DSE frameworks have notable limitations in both the pre-training stage and the adaptation stage.

1) *Upstream Pre-training Issues*: In the pre-training stage, the surrogate model is prone to overfitting the pre-training data and faces data ambiguity. For instance, previous methods [12]–[14] typically utilize machine learning based surrogate models, such as tree-based ones. These models tend to capture patterns specific to the source workloads and lead to overfitting with standard supervised training. However, due to the variability and ambiguity inherent in cross-workload CPU DSE scenarios, similar architectural configurations may yield vastly different performance metrics across workloads. This discrepancy leads the model to learn misleading or unreliable features during pre-training. Consequently, these features may not generalize well when the model is fine-tuned on new target workloads, resulting in poor predictive accuracy and limited adaptability.

2) *Downstream Adaptation Issues*: For the downstream adaptation stage, the primary challenge lies in the method of knowledge transfer. Many prior works assume that workloads are similar [12], [15], [16], employing methods like Wasserstein distance [25] to compare metric space distributions between source and target workloads, but this assumption does not always hold. In real-world scenarios, workloads exhibit significant variability in computational behaviors, data access patterns, and performance requirements, making this assumption unreliable.

For instance, our experiments on the SPEC CPU 2017 benchmark suite (as shown in Fig. 2) demonstrate that the similarities across workloads are inconsistent, with many showing substantial differences. This variability undermines the effectiveness of transfer learning methods like TrEnDSE when faced with diverse and complex workload characteristics. As a result, these methods may struggle to deliver accurate performance predictions for real-world DSE tasks, underscoring the need for a more adaptive, workload-independent approach to tackle the challenges of cross-workload CPU DSE.

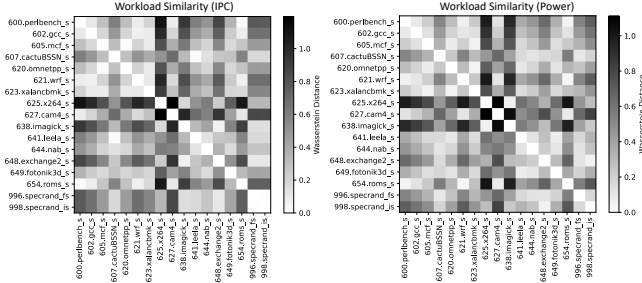


Fig. 2. Wasserstein distances among SPEC CPU 2017 workloads. The darker the color, the less similar the two workloads.

B. Rethinking Cross-workload CPU DSE

To overcome the limitations of prior arts, we introduce two innovative solutions tailored to the distinct stages.

For the pre-training stage, we address the challenges of overfitting and data ambiguity by reframing the task as a few-shot meta-learning problem. This perspective shift allows the model to be trained on a variety of tasks using only a small number of labeled samples, thereby reducing the risk of overfitting and improving its ability to generalize across different workloads. By leveraging the meta-learning technique, the surrogate model can capture shared patterns across workloads, eliminating the impact of data ambiguity.

For the adaptation stage, we recognize the limitations of conventional methods that rely on workload similarity for knowledge transfer. Instead, we propose shifting the focus from workload similarity to the intrinsic properties of the architectural design. To harness these inherent characteristics, we introduce the WAM algorithm. WAM uncovers key relations between architectural parameters and leverages them during adaptation. This novel approach enhances the model's robustness and adaptability, improving the overall effectiveness of cross-workload CPU DSE.

IV. DESIGN OF METADSE

A. Overview of MetaDSE

In this section, we introduce MetaDSE, a framework designed to enhance the efficiency of cross-workload CPU DSE. The workflow of MetaDSE, illustrated in Fig. 3, is divided into two main stages: pre-training (①–④) and adaptation (①–③). For the surrogate model f , we employ a transformer-based predictor, inspired by the strong performance of AttentionDSE [7] in CPU DSE tasks.

In the pre-training stage, the meta-training process begins by creating a copy of the surrogate model (parameterized by $\hat{\theta}$). This model is first fine-tuned on the support set of each task through a few gradient descents (①, ②). Subsequently, the fine-tuned model evaluates the loss on the corresponding query set (③), and this computed loss contributes to the overall meta-loss (④). The meta-loss is then used to update the parameters of the original surrogate model (parameterized by θ). After each training epoch, a meta-validation step is conducted to evaluate the model's performance and to identify the optimal parameters for downstream tasks. In the adaptation stage, the pre-trained surrogate model (parameterized by $\hat{\theta}^*$) undergoes fine-tuning with the WAM algorithm (②). WAM focuses on the inherent properties of the architecture, adjusting the model to emphasize relevant interactions between architectural parameters. Further technical details on the WAM adaptation process are elaborated in the subsequent sections.

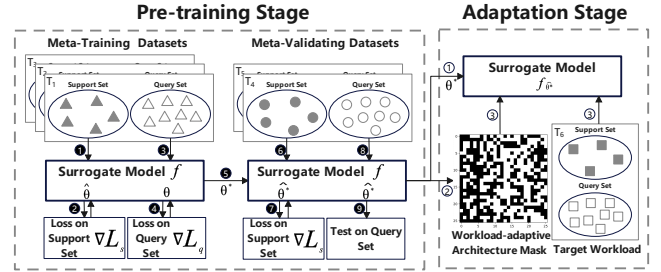


Fig. 3. The overview workflow of MetaDSE.

B. MAML-based Pre-training Stage

The pre-training stage in MetaDSE typically aims to build a model that performs well across a wide range of tasks. Instead, our MAML-based approach focuses on finding an optimal model initialization, θ , which can rapidly adapt to new downstream tasks using few data. This strategy allows the predictor to generalize effectively to diverse workloads without extensive retraining, tackling the core challenge of the few-shot problem in cross-workload CPU DSE.

Algorithm 1 describes the MAML-based pre-training process for the surrogate model. Before the training stage, each iteration begins by sampling tasks from the task distribution $P(T)$, representing different workloads. For each task, the model is trained using a support set S and evaluated on a query set Q . The training in MAML can be divided into two phases: the inner loop and the outer loop. The inner loop (from line 4 to line 12) focuses on task-specific adaptation, simulating the fine-tuning process during model deployment. Here, the model updates its parameters using the support set from a

Algorithm 1: The Pre-Training Procedure of Predictors

Input: f_{θ} : the predictor with random parameters; $P(T)$: task distribution of one workload; s, q : sample amount of support set and query set in one task, correspondingly; α, β : learning rate in inner and outer loop;

Output: f_{θ^*} : the meta-trained predictor

```

1 randomly initialize  $\theta$ 
2 while not done do
3   Sample tasks from some workloads' distribution  $T_i \sim P(T)$ 
4   for each task  $t$  in  $T_i$  do
5      $\hat{\theta} = \theta$ 
6     Split out support set  $S = (x_j, y_j)$  and query set  $Q = (x_k, y_k)$  from  $t$ ,  $S, Q \leftarrow \text{Spilt}(t, s, q)$ 
7     for each inner step do
8       Evaluate  $\nabla_{\hat{\theta}} \mathcal{L}(f_{\hat{\theta}})$  with respect to  $S$ 
9       Compute adapted parameters with gradient:
10       $\hat{\theta} = \hat{\theta} - \alpha \nabla_{\hat{\theta}} \mathcal{L}(f_{\hat{\theta}})$ 
11    end
12    Save  $Q$  and  $f_{\hat{\theta}}$  temporarily
13  end
14  Evaluate and average meta gradients of every task from  $T_i$ ,
15   $\nabla_{\theta} \mathcal{L}(f_{\hat{\theta}})$  with respect to each  $Q$  in every  $t$ 
16  Compute adapted meta parameters with meta gradient:
17   $\theta = \theta - \beta \nabla_{\theta} \mathcal{L}_{T_i}(f_{\hat{\theta}})$ 
18 end
19  $\theta^* = \theta$ 
20 return final predictor  $f_{\theta^*}$ 

```

specific task. This phase aims to find temporary parameters $\hat{\theta}$ that adapt well to the current task. The update rule in this phase is: $\hat{\theta} \leftarrow \hat{\theta} - \alpha \nabla_{\hat{\theta}} \mathcal{L}(\hat{\theta})$, where α is the inner loop learning rate. This gradient descent adjusts the parameters to minimize the task-specific loss $\mathcal{L}$, effectively mimicking how the model would adapt when exposed to new data. The outer loop (from line 2 to line 15) aims to optimize the initial parameters θ by aggregating the feedback from multiple tasks. After the inner loop updates for each task, the model evaluates its performance on the query set to calculate the meta-loss. This meta-loss reflects how well the adapted model (with parameters $\hat{\theta}$) generalizes across different tasks. The meta-parameters θ are then updated based on the meta-gradient: $\theta = \theta - \beta \nabla_{\theta} \mathcal{L}_{T_i}(f_{\hat{\theta}})$ where β is the meta-learning rate. This outer-loop update ensures that the initial parameters θ are optimized to quickly adapt to new tasks with minimal data.

This two-level optimization process enables MetaDSE to efficiently adapt to new workloads. It ensures the pre-trained model generalizes well to diverse, unseen tasks, minimizing the need for extensive fine-tuning on large datasets.

C. Workload-adaptive Architectural Mask Adaptation Stage

In cross-workload CPU DSE tasks, workload dissimilarity poses a significant challenge for knowledge transfer. To tackle this, we propose WAM adaptation, which leverages inherent architectural properties to enhance the pre-trained model's generalization to unseen workloads.

A critical issue is that not all architectural parameters are fully correlated. Previous efforts [12]–[15], [18] have largely overlooked this, leading to noise from irrelevant parameter interactions and reduced prediction accuracy. To address this, we introduce an algorithm that generates a mask for the surrogate model, utilizing intermediate attention weights from the transformer-based predictor during pre-training. This mask

Algorithm 2: The Adaptation Procedure of Predictors

Input: f_{θ^*} : the meta-trained predictor; $P(T)$: task distribution of test workload; s, q : sample amount of support set and query set in one task, correspondingly; γ : learning rate during adaptation; M : workload-adaptive architectural mask.

Output: $f_{\hat{\theta}^*}$: the adapted predictor

```

1 Equip  $f_{\theta^*}$  with  $M$  in its self-attention operator
2 Set  $M$  to learnable state:  $M.\text{required\_grad} = \text{True}$ 
3 Sample tasks from test workload's distribution  $T_i \sim P(T)$ 
4 for each task  $t$  in  $T_i$  do
5    $\hat{\theta}^* = \theta^*$ 
6   Split out support set  $S = (x_j, y_j)$  and query set  $Q = (x_k, y_k)$  from  $t$ ,  $S, Q \leftarrow \text{Spilt}(t, s, q)$ 
7   for each step do
8     Evaluate  $\nabla_{\hat{\theta}^*} \mathcal{L}(f_{\hat{\theta}^*}, M)$  with respect to  $S$ 
9     Compute adapted parameters with gradient:
10     $\hat{\theta}^* = \hat{\theta}^* - \gamma \nabla_{\hat{\theta}^*} \mathcal{L}(f_{\hat{\theta}^*})$ 
11   end
12   return adapted predictor  $f_{\hat{\theta}^*}$ 
13 end

```

filters out irrelevant parameter interactions, enabling the model to focus on key factors and improving prediction accuracy.

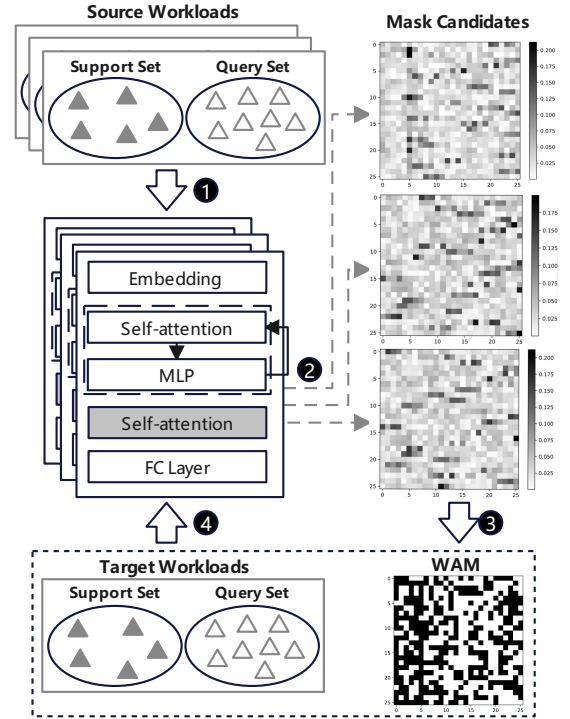


Fig. 4. The example of workload-adaptive architectural mask generation.

As Fig. 4 shows, we utilize attention weights obtained during the pre-training process to analyze the relationships between architectural parameters (1, 2). We extract all the attention weights from the last self-attention layer, referring to them as mask candidates. By identifying parameter interactions that frequently occur across diverse workloads, we construct a WAM (3) that highlights high-frequency correlations, effectively filtering out less relevant interactions.

After the mask is generated, we leverage the mask and the target workloads to the adaptation stage (4). The procedure is illustrated in Algorithm 2. The process also begins by sampling

tasks from the test workload’s distribution $P(T)$. For each sampled task t , we initialize the adapted model parameters as $\hat{\theta}^* = \theta^*$ and split the task data into a support set S and a query set Q . During the adaptation process, we iteratively update the model parameters $\hat{\theta}^*$ using gradients computed based on the loss $\mathcal{L}$ evaluated on the support set S . The update rule $\hat{\theta}^* = \hat{\theta}^* - \gamma \nabla_{\hat{\theta}^*} \mathcal{L}(f_{\hat{\theta}^*})$ ensures that the model learns with mask M to minimize the error on the specific task at hand. Finally, the adapted predictor $f_{\hat{\theta}^*}$ is returned for each task, incorporating the updated parameters that better capture the features of target workloads.

WAM adaptation allows the surrogate model to focus on critical parameter interactions, reducing noise from irrelevant features and enhancing prediction accuracy. Moreover, it introduces a novel approach to knowledge transfer, prioritizing architectural properties over workload similarity.

V. EXPERIMENTAL SETUP

Datasets Generation. We utilize GEM5 [28] as the timing-accurate simulator and McPAT [29] as the power modeling tool. We extend GEM5 using Python to generate different CPU cores with varying configurations. To bolster the credibility of our experiments, we employ SPEC CPU 2017 [30] as the evaluation benchmark. We utilize Simpoints [31] for each workload evaluation. Each workload is divided into at most 30 clusters, with each cluster containing ten million instructions.

For dataset splitting, we iteratively and randomly designated seven datasets for training, five for validation, and five for testing. This approach ensured balanced and robust training across various dataset combinations.

Baselines. We evaluate our proposed MetaDSE framework against several baselines from different perspectives. The first baseline is TrEnDSE [12], a state-of-the-art cross-workload CPU DSE framework. We then introduce TrEnDSE-Transformer, where the ensemble model in TrEnDSE is replaced with a Transformer-based predictor. Lastly, MetaDSE-w/o WAM is a variant of MetaDSE that excludes the WAM adaptation.

The Explored Design Space. The design space of the out-of-order processor is listed in Table I. The values in the third column are formatted as “start number: end number: stride”. A two-level cache hierarchy with an 8192MB DRAM main memory configuration is employed.

Evaluation Metrics. The accuracy of the model is primarily measured using the Root Mean Squared Error (RMSE), which is calculated as:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2} \quad (1)$$

where y_i is the actual value, $\hat{y}_i$ is the predicted value, and N is the number of data points.

Additionally, other metrics such as Mean Absolute Percentage Error (MAPE) and Explained Variance (EV) are used for a more comprehensive evaluation:

TABLE I
MICROARCHITECTURE DESIGN SPACE SPECIFICATION.

Parameters	Description	Candidate Value
Core Frequency	the frequency of CPU core in GHz	1/1.5/2/2.5/3
Pipeline Width	fetch/decode/rename/dispatch/issue/writeback/commit width	1:12:1
Fetch Buffer	fetch buffer size in bytes	16/32/64
Fetch Queue	fetch queue size in μ -ops	8:48:4
Branch Predictor	predictor type	BiModeBP/TournamentBP
RAS Size	return address stack size	16:40:2
BTB Size	branch target buffer size	1024/2048/4096
ROB Size	reorder buffer entries	32:256:16
Int/Fp RF Number	number of physical integer/floating-point registers	64:256:8
Inst Queue	number of instruction queue entries	16:80:8
Load/Store Queue	number of load/store queue entries	20:48:4
IntALU	number of integer ALUs	3:8:1
IntMultDiv	number of integer multipliers and dividers	1:4:1
FpALU	number of floating-point ALUs	1:4:1
FpMultDiv	number of floating-point multipliers and dividers	1:4:1
Cacheline	cacheline size	32/64
L1 Cache Size	size of ICache in KB	16/32/64
L1 Cache Assoc.	associative sets of ICache	2/4
L2 Cache Size	size of L2 Cache in KB	128/256
L2 Cache Assoc.	associative sets of L2 Cache	2/4

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left(\frac{|y_i - y_i^{\text{real}}|}{y_i^{\text{real}}} \right) \times 100. \quad (2)$$

$$\text{EV} = 1 - \frac{\sum_{i=1}^N (y_i - \bar{y})^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (3)$$

where $\bar{y}$ is the mean of the actual values. Lower RMSE and MAPE values, and higher EV values, indicate better model performance.

VI. EXPERIMENT RESULTS

A. Overall Analysis

The implementation of MetaDSE has two stages: upstream pre-training and downstream WAM adaptation. For pre-training, we run 15 epochs with 200 tasks per workload, each task having five support samples and forty-five query samples. Inner and outer loop learning rates are 1×10^{-5} and 1×10^{-4} , respectively. In each iteration, every model copy is optimized via five inner-loop steps on the support set using SGD, and the accumulated meta-loss is used to update the original model with the Adam optimizer. In the downstream stage, we fine-tune the pre-trained model using ten gradient steps with a

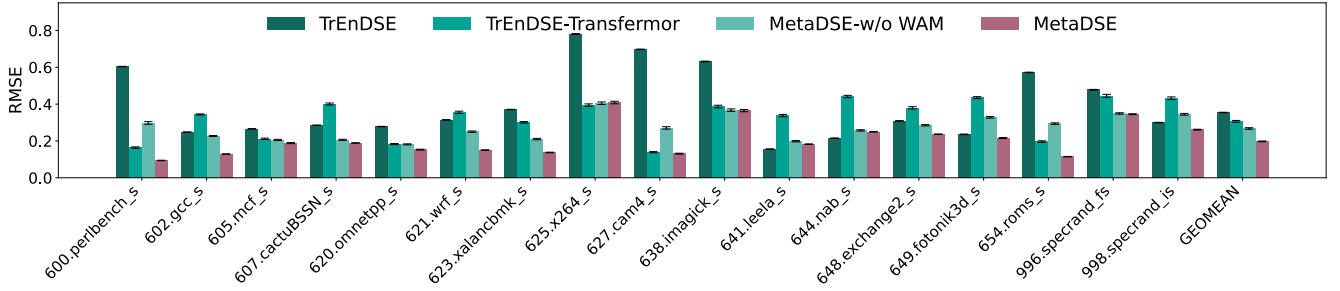


Fig. 5. Comparison of IPC RMSE per workload on SPEC CPU 2017 with the SOTA cross-workload CPU DSE framework (lower is better for RMSE).

TABLE II

THE OVERALL RESULTS, AVERAGED ACROSS FIVE DATASETS (600.PERLBENCH_S, 605.MCF_S, 620.OMNETPP_S, 623.XALANCBMK_S, AND 627.CAM4_S), ARE EVALUATED USING RMSE, MAPE, AND EXPLAINED VARIANCE.

Models / Metrics	RMSE ↓		MAPE ↓		EV ↑	
	IPC	Power	IPC	Power	IPC	Power
RF	0.4389±0.0150	0.5344±0.0198	1.1624±0.0827	0.3356±0.0178	-0.7997±0.1231	0.4470±0.0509
GBRT	0.3637±0.0125	0.4539±0.0171	0.9486±0.0678	0.2667±0.0144	-0.5152±0.1137	0.4634±0.0529
TrEnDSE	0.3270±0.0109	0.3990±0.0146	0.8386±0.0595	0.2348±0.0123	-0.5142±0.1079	0.5711±0.0410
MetaDSE	0.2204±0.0072	0.3969±0.0095	0.5909±0.0460	0.2330±0.0045	-0.0471±0.0056	0.3189±0.0065

learning rate of 1×10^{-5} and cosine annealing. We evaluate 1000 tasks per workload for mean and confidence intervals.

Fig. 5 illustrates the prediction accuracy comparison between MetaDSE and various baseline models on the SPEC CPU 2017 dataset. The last column presents the geometric mean (GEOMEAN) of prediction errors across all tasks. MetaDSE achieves a significant reduction in prediction error, lowering it by 44.3% compared to the state-of-the-art cross-workload CPU DSE framework, TrEnDSE. When comparing TrEnDSE with TrEnDSE-Transformer, we observe that simply replacing the ensemble learning model with a more powerful Transformer model provides some performance improvement. However, this improvement is relatively modest and inconsistent across different tasks. In contrast, within the meta-learning-based DSE framework, the inclusion of our proposed WAM adaptation technique leads to a substantial performance boost, reducing the average prediction error by 27%, and highlighting the effectiveness of adapting to workload-specific characteristics.

We further compare MetaDSE with several commonly used models in transfer learning for predicting IPC and power, including Random Forest (RF) and Gradient Boosting Regression Trees (GBRT), shown in Table II. Our results reveal that MetaDSE consistently outperforms TrEnDSE and other baselines across all evaluated workloads, demonstrating lower RMSE and MAPE values while achieving higher EV scores. These results further highlight the superiority of MetaDSE over traditional transfer learning approaches, emphasizing its enhanced ability to generalize and adapt effectively across different workloads in cross-workload CPU DSE tasks.

B. Exploring Data Impact

We further investigate the impact of pre-training and adaptation data on MetaDSE performance.

1) *Sensitivity of Pre-training:* For the upstream pre-training stage, we fix the downstream adaptation support set size to ten and change the support set size of the pre-training stage from five to forty. Fig. 6 illustrates that when the upstream and downstream data are aligned, the model achieves optimal transfer performance, with the highest EV and lowest RMSE. This improvement is attributed to the matched knowledge pattern and the higher learning efficiency when the data distribution is consistent across pre-training and adaptation phases, leading to better generalization and model adaptation.

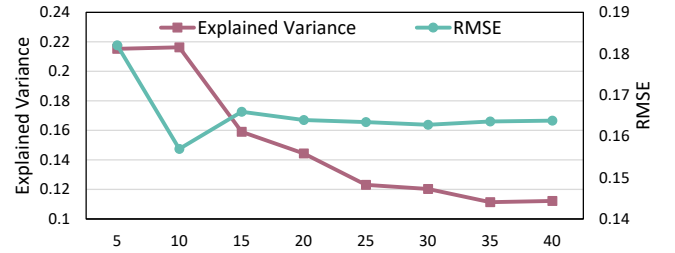


Fig. 6. The variation in explained variance and RMSE with respect to changes in the source sample size.

2) *Sensitivity of Adaptation:* We further investigate the impact of downstream adaptation data and compare MetaDSE to commonly used machine learning models, including RF, GBRT, and TrEnDSE. In this experiment, we fix the support set size of the upstream pre-training stage to ten while varying the adaptation support set size from five to forty. The results presented in Table III show that when the pretraining data size is fixed, increasing the adaptation data size results in modest performance improvements for MetaDSE. Notably, MetaDSE achieves high performance even with a smaller amount of adaptation data.

TABLE III
RMSE ON IPC PREDICTION AS THE DOWNSTREAM ADAPTATION SUPPORT
SIZE (K) CHANGES.

Models / K	5	10	20	30	40
RF	0.4409	0.4397	0.4390	0.4386	0.4380
GBRT	0.2577	0.2390	0.2356	0.2321	0.2299
Baseline	0.2616	0.2397	0.2229	0.2147	0.2076
MetaDSE	0.1580	0.1562	0.1485	0.1471	0.1466

VII. CONCLUSION

In this work, we introduce meta-learning to cross-workload CPU DSE tasks and propose MetaDSE. MetaDSE utilizes the MAML and proposes the WAM algorithm to uncover the inherent architectural properties, offering a novel approach to prior knowledge transfer. Experiments on SPEC CPU 2017 benchmarks demonstrate that MetaDSE significantly reduces prediction errors compared to state-of-the-art methods, highlighting its effectiveness in improving model adaptability.

ACKNOWLEDGMENT

This work was supported by National Key Research and Development Program (Grant No. 2022YFB4501400), the National Natural Science Foundation of China (Grant No. 62202451 and No. 62302477), CAS Project for Young Scientists in Basic Research (Grant No. YSBR-029), and CAS Project for Youth Innovation Promotion Association.

REFERENCES

- [1] B. C. Lee and D. M. Brooks, "Illustrative design space studies with microarchitectural regression models," in *2007 IEEE 13th International Symposium on High Performance Computer Architecture*, pp. 340–351, IEEE, 2007.
- [2] A. Deshwal, S. Belakaria, G. Bhat, J. R. Doppa, and P. P. Pande, "Learning pareto-frontier resource management policies for heterogeneous socs: An information-theoretic approach," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pp. 607–612, IEEE, 2021.
- [3] D. Li, S. Yao, Y.-H. Liu, S. Wang, and X.-H. Sun, "Efficient design space exploration via statistical sampling and adaboost learning," in *Proceedings of the 53rd Annual Design Automation Conference*, pp. 1–6, 2016.
- [4] D. Wang, M. Yan, Y. Teng, D. Han, X. Ye, and D. Fan, "A high-accurate multi-objective ensemble exploration framework for design space of cpu microarchitecture," in *Proceedings of the Great Lakes Symposium on VLSI 2023, GLSVLSI '23*, (New York, NY, USA), p. 379–383, Association for Computing Machinery, 2023.
- [5] G. Mariani, A. Brankovic, G. Palermo, J. Jovic, V. Zaccaria, and C. Silvano, "A correlation-based design space exploration methodology for multi-processor systems-on-chip," in *Proceedings of the 47th Design Automation Conference, DAC '10*, (New York, NY, USA), p. 120–125, Association for Computing Machinery, 2010.
- [6] H. Wang, J. Shi, and Z. Zhu, "An expected hypervolume improvement algorithm for architectural exploration of embedded processors," in *Proceedings of the 53rd Annual Design Automation Conference*, pp. 1–6, 2016.
- [7] R. Xue, H. Wu, M. Yan, Z. Xiao, X. Ye, and D. Fan, "Multi-objective optimization in cpu design space exploration: Attention is all you need," 2024.
- [8] H. Fan, Y. Wang, S. Li, T. Liang, and W. Zhang, "Explainable fuzzy neural network with multi-fidelity reinforcement learning for micro-architecture design space exploration," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, pp. 1–6, 2024.
- [9] D. Wang, M. Yan, X. Liu, M. Zou, T. Liu, W. Li, X. Ye, and D. Fan, "A high-accurate multi-objective exploration framework for design space of cpu," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6, 2023.
- [10] C. Bai, Q. Sun, J. Zhai, Y. Ma, B. Yu, and M. D. Wong, "Boom-explorer: Risc-v boom microarchitecture design space exploration framework," in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pp. 1–9, IEEE, 2021.
- [11] C. Bai, Q. Sun, J. Zhai, Y. Ma, B. Yu, and M. D. Wong, "Boom-explorer: Risc-v boom microarchitecture design space exploration," *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 1, pp. 1–23, 2023.
- [12] D. Wang, M. Yan, Y. Teng, D. Han, H. Dang, X. Ye, and D. Fan, "A transfer learning framework for high-accurate cross-workload design space exploration of cpu," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pp. 1–9, 2023.
- [13] D. Li, S. Wang, S. Yao, Y.-H. Liu, Y. Cheng, and X.-H. Sun, "Efficient design space exploration by knowledge transfer," in *2016 International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, pp. 1–10, 2016.
- [14] D. Li, S. Yao, S. Wang, and Y. Wang, "Cross-program design space exploration by ensemble transfer learning," in *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pp. 201–208, 2017.
- [15] S. Khan, P. Xekalakis, J. Cavazos, and M. Cintra, "Using predictive modeling for cross-program design space exploration in multicore systems," in *16th International Conference on Parallel Architecture and Compilation Techniques (PACT 2007)*, pp. 327–338, 2007.
- [16] Q. Guo, T. Chen, Y. Chen, L. Li, and W. Hu, "Microarchitectural design space exploration made fast," *Microprocessors and Microsystems*, vol. 37, no. 1, pp. 41–51, 2013.
- [17] Y. Ding, N. Mishra, and H. Hoffmann, "Generative and multi-phase learning for computer systems optimization," in *Proceedings of the 46th International Symposium on Computer Architecture*, pp. 39–52, 2019.
- [18] C. Dubach, T. M. Jones, and M. F. O'Boyle, "An empirical architecture-centric approach to microarchitectural design space exploration," *IEEE Transactions on Computers*, vol. 60, no. 10, pp. 1445–1458, 2010.
- [19] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *International conference on machine learning*, pp. 1126–1135, PMLR, 2017.
- [20] M. Ren, E. Triantafillou, S. Ravi, J. Snell, K. Swersky, J. B. Tenenbaum, H. Larochelle, and R. S. Zemel, "Meta-learning for semi-supervised few-shot classification," *arXiv preprint arXiv:1803.00676*, 2018.
- [21] Q. Sun, Y. Liu, T.-S. Chua, and B. Schiele, "Meta-transfer learning for few-shot learning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 403–412, 2019.
- [22] L. Torrey and J. Shavlik, "Transfer learning," in *Handbook of research on machine learning applications and trends: algorithms, methods, and techniques*, pp. 242–264, IGI global, 2010.
- [23] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on knowledge and data engineering*, vol. 22, no. 10, pp. 1345–1359, 2009.
- [24] D. A. Reynolds *et al.*, "Gaussian mixture models," *Encyclopedia of biometrics*, vol. 741, no. 659–663, 2009.
- [25] S. Vallender, "Calculation of the wasserstein distance between probability distributions on the line," *Theory of Probability & Its Applications*, vol. 18, no. 4, pp. 784–786, 1974.
- [26] X. Luo, H. Wu, J. Zhang, L. Gao, J. Xu, and J. Song, "A closer look at few-shot classification again," in *International Conference on Machine Learning*, pp. 23103–23123, PMLR, 2023.
- [27] J. Yang, H. Wu, J. Zhang, L. Gao, and J. Song, "Effective and efficient few-shot fine-tuning for vision transformers," in *2024 IEEE International Conference on Multimedia and Expo (ICME)*, pp. 1–6, IEEE, 2024.
- [28] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 simulator," *SIGARCH Comput. Archit. News*, vol. 39, p. 1–7, aug 2011.
- [29] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, "Mcpat: An integrated power, area, and timing modeling framework for multicore and manycore architectures," in *Proceedings of the 42nd annual ieee/acm international symposium on microarchitecture*, pp. 469–480, 2009.
- [30] Standard Performance Evaluation Corporation, "Spec cpu 2017," <https://www.spec.org/cpu2017/>, 2022.
- [31] G. Hamerly, E. Perelman, J. Lau, and B. Calder, "Simpoint 3.0: Faster and more flexible program phase analysis," *Journal of Instruction Level Parallelism*, vol. 7, no. 4, pp. 1–28, 2005.